If Everything Is "Internet-Reachable," Nothing Is.

Matthew Venne
Chief Technology Officer, stackArmor

July 2026

A plain-language companion to "A Finding-Level Model for Internet Reachability under the FedRAMP VDR/VER Rules."

The problem in one sentence

FedRAMP's new rules make one question — *is this vulnerability reachable from the internet?* — a major driver of how fast you must fix it, and then define "reachable" so broadly that, read to the letter, almost everything in a connected system qualifies.

Follow the words literally and follow the data. The request hits the application; the application writes the database; the database replicates; events land on the message bus; workers consume them; logs collect all of it. Every one of those components "might receive a payload from the internet," so every flaw on every one of them lands in the internet-reachable bucket. And a label that everything carries means nothing: it cannot tell your security team what to fix first, and it puts your whole estate on the fastest remediation clocks at once — at the top of the matrix, that is a two-day clock, everywhere, forever, plus per-finding documentation to match. No provider of realistic size can run that way, and nothing in the rules suggests they were designed to be unsatisfiable.

The way out is not to argue with the definition. It is to notice what else FedRAMP's own text says, and to make the determination a computation instead of a label.

What the words actually say

Two details in the rule text are the source of the breadth.

First, **reachability follows the data, not the network diagram.** The rule explicitly includes systems that "have no direct route to/from the internet but receive payloads or otherwise take action triggered by internet activity." Log4Shell made this vivid: the classic victim was a thumbnail generator or log indexer on a private subnet, no public IP, often no listening port at all. It just parsed strings that internet users had supplied. Under FedRAMP's definition, that box was internet-reachable the whole time.

Second, **reachability is a property of the vulnerability, not the server.** The rule says it "applies only to the specific vulnerable machine-based information resources processing the payload." The same host can carry one flaw that is internet-reachable and another that is not.

But the same rule text carries a third detail most readings skip. FedRAMP's notes say the simplest way to prevent exploitation is to "intercept, inspect, filter, *sanitize*, reject, or otherwise

deflect" the triggering payload before the vulnerable code processes it — and that "once this prevention is in place the vulnerability should no longer be considered an internet-reachable vulnerability." *Sanitize* is FedRAMP's own word. And sanitizing internet input is not some exotic control: parameterized queries, schema validation, type and bounds checks, output encoding — this is what production applications already do, every day, before handing data to the tiers behind them.

Compute it per finding

The paper replaces the label with an equation, and the equation fits in a sentence. For each finding, build two lists: **what the internet can actually deliver to this component** (which doors are open, which protocols survive the load balancers and firewalls in between, what the tiers in front clean out), and **what this particular flaw needs in order to fire** (a network path, a protocol, a payload class). **If the lists overlap, the finding is internet-reachable. If they don't, it isn't.** Every entry in both lists must be backed by evidence, and any entry nobody can pin down counts as an overlap — so missing evidence always errs toward "reachable," never away from it.

The asset side of the list is computed by five deterministic gates — routing, firewall source attribution, edge authentication, Kubernetes exposure, and process-to-port mapping — and then pushed through every hop on the way in: firewalls drop what they block, load balancers rewrite what they translate, login proxies convert public traffic to authenticated traffic, and **sanitizing tiers strip the payload classes they neutralize** from everything downstream. That last hop is the one the blanket reading ignores, and it is the one FedRAMP's own prevention note endorses.

The evidence is the auditor's to collect — and most of it already exists

Here is the assertion that changes the day-to-day picture: verifying that your application really does sanitize its internet input is *assessment work*. It is the auditor's job to collect the evidence — and the evidence is not exotic paperwork, it is the artifacts that already exist. Code review records over the input-handling paths. Static analysis (SAST) results. Unit and integration tests that fire hostile payloads and show them handled inertly. And — decisively — the regular web-application (DAST) scanning that FedRAMP continuous monitoring *already requires*: those scans exist precisely to throw injection and malformed-content payloads at the running application and report what got through. A clean recurring web-app scan over the endpoints that feed your database *is* sanitization evidence, refreshed on the cadence FedRAMP already mandates.

So no new documentation regime is needed. The 3PAO scopes which scans, tests, and reviews cover which internal hand-offs; the agency or FedRAMP can ask for the same evidence at any time. And the scrutiny this buys goes where the risk actually concentrates: **on the components that process data from unauthenticated users** — the login page, the public API, the upload endpoint, and the parsers their raw input feeds. Those see the internet's content before any cleaning has had its chance. The tiers behind them see only what survives.

The steady state: the reachable list looks like the internet-facing list

Put those pieces together and something practical falls out. Because the evidence collection is standing — assessments collect it, the required scans regenerate it — a conforming provider ends up,

for most reporting scenarios, with an internet-reachable list that converges on the internet-*accessible* list: the entry points, and the components handling raw unauthenticated input. Not because the two words mean the same thing (they don't, and the paper is strict about the vocabulary), but because standing proof of sanitization keeps closing the gap that the transitive definition opens.

That is what proportionality looks like: the urgent list is short, real, and defensible, and every "not reachable" behind it traces to evidence an assessor collected rather than a judgment call someone typed into a spreadsheet.

The day the calm breaks

The convergence has exactly one condition: attackers using known tricks. Every sanitization verdict is a claim about the techniques the world knows — the scans fire the payloads the industry knows to fire, the tests test the tricks the developers knew to test. A zero-day that defeats the known techniques dissolves those claims wholesale, and this is precisely the day FedRAMP's sweeping definition was written for.

Log4Shell, again, is the template. Before December 2021, nobody's sanitization guidance treated a JNDI lookup string inside a log line as dangerous — so no evidence collected before that day said anything about it. The moment the technique published, every downstream component whose logs could carry attacker-controlled strings was in play. And the response was a moving target: within days of the first WAF rules blocking the attack string, obfuscated variants sailed straight past them, and every vendor spent the month rewriting patterns in a game of whack-a-mole. When no patch is in place, yesterday's mitigation is only a hypothesis about tomorrow's traffic.

So the obligation on that day is due diligence, not paperwork: sweep for indicators of compromise everywhere the suspect data flows, put the known mitigations in, and keep watching whether they still hold — adjusting and adding as the technique evolves — until a patch exists and is applied. Whether you re-label the affected findings in your internal tracker is your call; the model doesn't quibble over ledger mechanics. Doing the work is not optional, and each suspended "not reachable" verdict is earned back only when fresh evidence shows your cleaning stops the new trick too.

THE WHOLE MODEL

Day to day: five gates find your true entry points; the auditor collects standing proof — required web-app scans, code reviews, tests — that your tiers clean what they pass along; and the internet-reachable list converges to the internet-facing list, with the deep scrutiny reserved for whatever handles input from strangers who never logged in. The day threat intel drops a technique that beats everyone's cleaning: assume it travels, hunt for compromise everywhere the data flows, mitigate and keep re-mitigating until the patch lands, and re-earn the exclusions with fresh evidence. FedRAMP's sweeping definition is written for that second day — not for every day.

The same flaw, two very different verdicts

The per-finding grain is the point, so here it is in action:

- **SQL injection flagged on a private database behind the application tier** — FedRAMP's own canonical case. No public IP, no internet route. Under the blanket reading: reachable, tightest clock. Under the model: the assessment has the data-access code review on record, static analysis is clean for injection on those paths, and the required web-app scans fire injection payloads at the API and show them handled inertly. The cleaning hop strips injection delivery from everything downstream: **not internet-reachable**, with the evidence bundle as the audit

artifact. Same database, no evidence collected? **Reachable, full stop** — the default doing its job.

- **An SSH flaw on a public web server:** the most internet-facing box in the estate — but port 22 is locked to the ops team's attributed addresses, and the flaw only fires over SSH, which internet data never arrives on. **Not internet-reachable**, on the most public host you own.

Opposite verdicts, and neither is decided by where the server sits — one is decided by evidence of cleaning, the other by evidence of what the doors actually deliver.

"But we have a WAF" — the attest-vs-downgrade trade

If in-code sanitization can end reachability, surely a WAF rule that blocks the exploit does too? FedRAMP does permit that reclassification. The paper's answer: you may, but you shouldn't — attest instead. Since "keep it classified reachable" sounds like leaving money on the table, it is worth walking through what each choice actually buys.

Start with the surprise: the downgrade buys you no schedule. The clock relief you want comes from FedRAMP's mitigation definitions, not from the reclassification. FedRAMP defines mitigation over *likelihood or impact*, and a WAF rule that verifiably blocks the triggering payloads is a likelihood reduction: partial mitigation moves the finding to a longer clock, and a rule that verifiably blocks *every* variant drives the likelihood to negligible — the finding is *fully mitigated* and leaves the remediation treadmill for routine operations. You get all of that while the finding is still classified internet-reachable, with a signed VEX statement carrying the claim. So the two postures land on the same deadlines. The choice is not about speed; it is about what your records say — and when they lie.

What the attestation buys is truth that survives failure. A WAF is a bolt-on device: rules get tuned during incidents, flipped to detection-only, bypassed by obfuscation — Log4Shell's whack-a-mole was exactly this — and a CDN migration can quietly route around one. Now compare the two ledgers on the day that happens. The attested record still reads: internet-reachable, mitigated by *this named rule* — which is still true, and the mitigation claim points at exactly the control your monitoring should be watching. The downgraded record reads: not internet-reachable, slow clock — which became false the moment the rule failed, with nothing in the vulnerability record to say so and nothing prompting a re-evaluation. Same exposure in reality; only one set of books is still honest.

Three more dividends follow from the same design:

- **Your own risk picture stays real.** Keeping the finding classified reachable means your dashboards keep showing the true residual exposure — what you are actually betting on the WAF to hold — instead of laundering it out of view.
- **The zero-day drill becomes a query.** When threat intel says a new technique bypasses WAF protections, the attested estate answers in one filter: every VEX statement naming a WAF control is suspect — re-check those. The downgraded estate has to first *rediscover* which quiet NIRV verdicts were secretly WAF-shaped before it can even start. On the day the calm breaks, that difference is hours.
- **The audit artifact is simply better.** A signed, machine-readable statement naming the exact protecting control is something a 3PAO can verify and re-check; a silent absence from the urgent list is something they will question. And ongoing: the downgrade path obligates you

to re-validate a reachability call on *every* perimeter change — easy to promise, rarely done — while the attestation localizes that dependency in one revocable statement.

The contrast with in-code sanitization is what makes this a principled line rather than a bias against WAFs. Your application's input handling is shipped with the application, versioned with it, and re-tested on every build — its evidence renews itself at exactly the cadence the code changes, which is why it earns the exclusion directly. A perimeter device's health is a minute-to-minute operational fact, so the claim that depends on it belongs in a revocable attestation, not baked into the classification.

Two boundaries no mitigation moves. Known Exploited Vulnerabilities keep their CISA due dates "even if the vulnerability has been fully mitigated" — no virtual patch and no reclassification extends a KEV clock. And endpoint protection changes nothing about reachability: even a blocking-mode EDR that genuinely stops the exploit at runtime is evidence about exploitability and impact — worth real credit on those levers — not about whether internet data reaches the code.

The bottom line

Five claims carry the whole paper:

1. **Blanket "everything is reachable" is not a security posture.** A label everything carries prioritizes nothing and sets clocks nobody can keep.
2. **Reachability is a per-finding computation:** what the internet can deliver to this component, intersected with what this flaw needs to fire — every factor evidenced, every gap defaulting to "reachable."
3. **Sanitization is FedRAMP-named prevention, and verifying it is the auditor's job** — using evidence that already exists: code reviews, SAST, tests, and the DAST scanning continuous monitoring already requires. Extra scrutiny goes to whatever processes unauthenticated input.
4. **In the steady state, the reachable list converges on the internet-facing list** — proportionate, short, and defensible line by line.
5. **The broad definition is for the zero-day day.** When a technique defeats known sanitization and WAF protections, hunt for compromise, mitigate and re-mitigate until the patch lands, and re-earn the exclusions with fresh evidence — however you choose to track it internally.

The full paper has the equations with every symbol explained, the five gates, the evidence criteria, the worked examples, and the VEX mappings. But the idea you came for is simple: **stop treating FedRAMP's definition as a label that everything wears, and start treating it as a computation — one your auditors already have most of the evidence to run.**

Read the full technical paper: A Finding-Level Model for Internet Reachability under the FedRAMP VDR/VER Rules